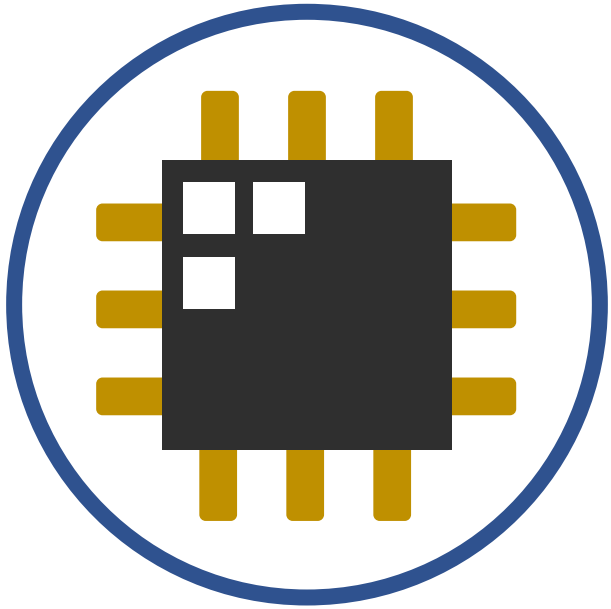
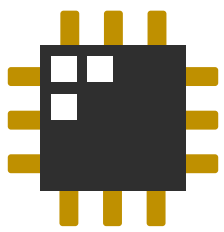




EEE 113:

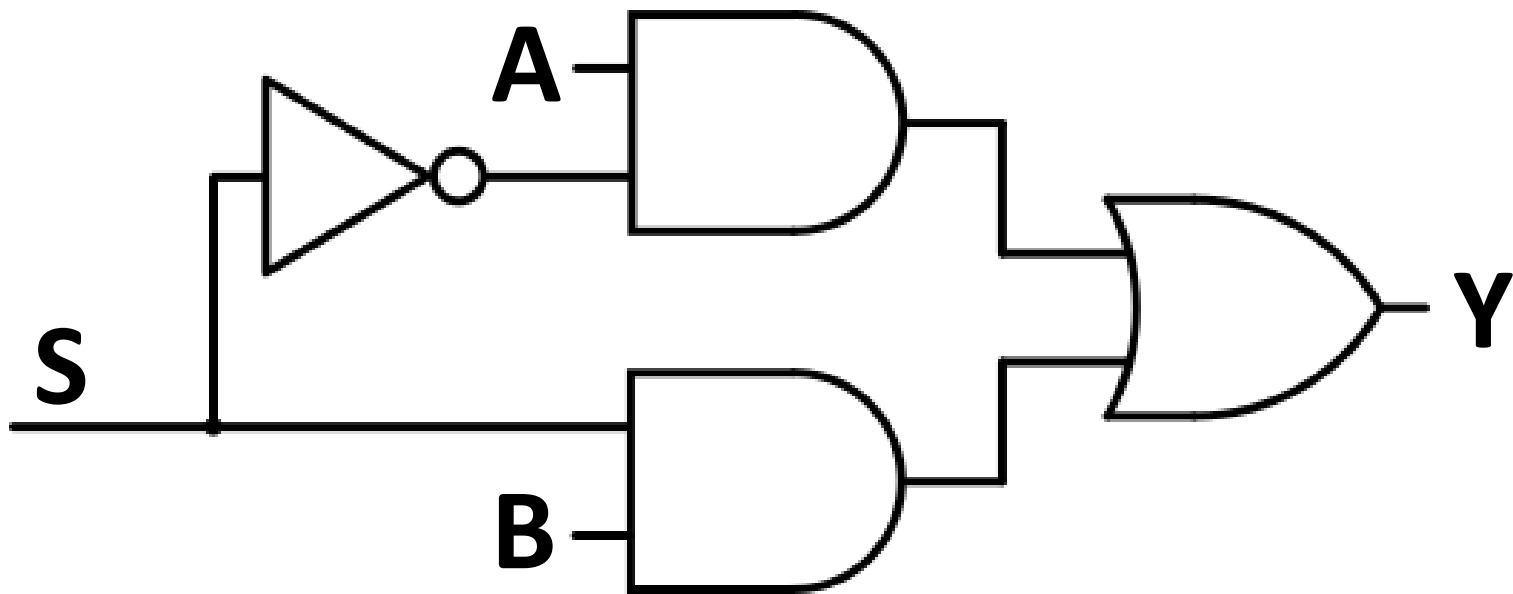
Introduction to Electrical and Electronics Engineering Systems

Week 2 – Multiplexers and Decoders



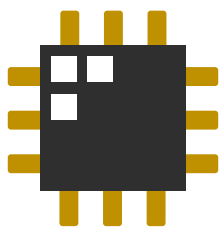
Remember this?

- We can make combinations of logic gates to make more complex logic!



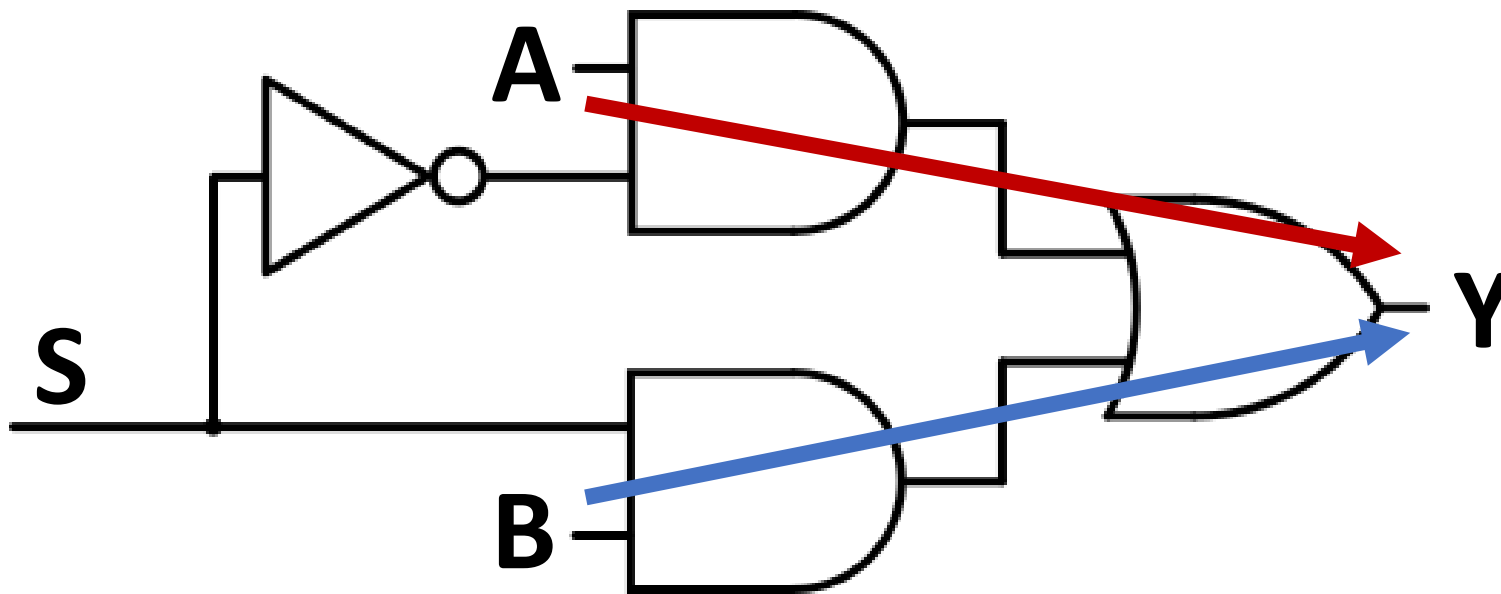
$$Y = S'A + SB$$

Inputs			Out
S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector

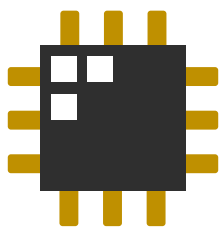


If $S = 0$, $Y = A$

If $S = 1$, $Y = B$

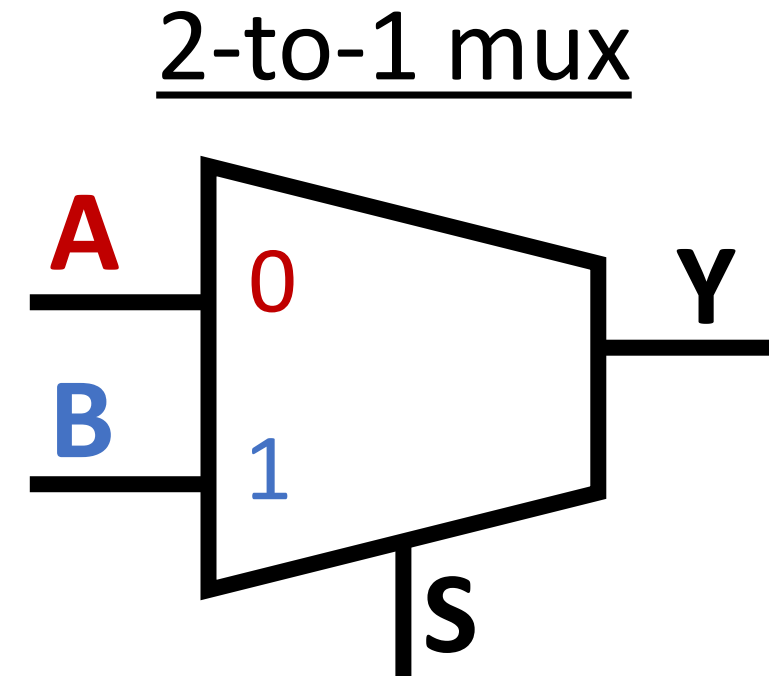
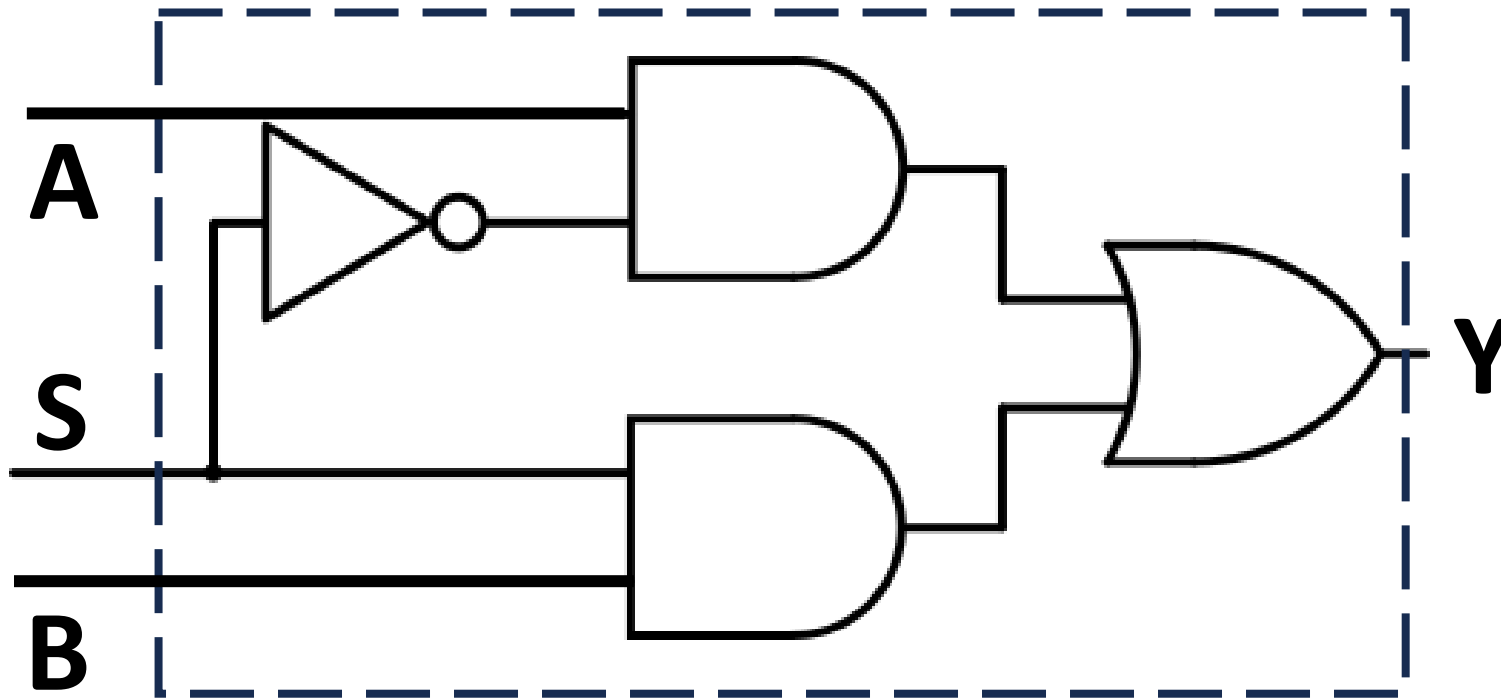
$$Y = S'A + SB$$

Inputs			Out
S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



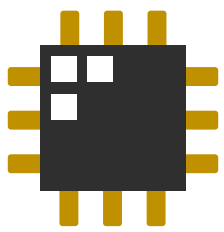
Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector



If $S = 0$, $Y = A$

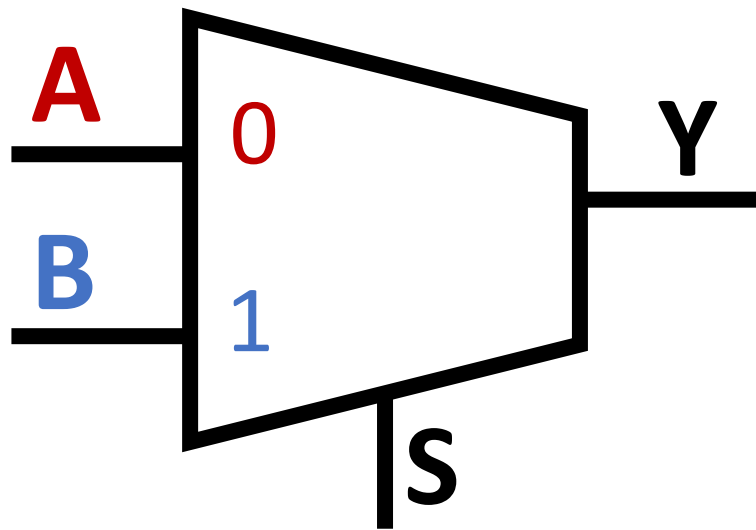
If $S = 1$, $Y = B$



Multiplexers

- Multiplexer (mux) is a circuit that selects data from *one of many* (2^n) inputs and directs it to a *single* output using an n-bit selector

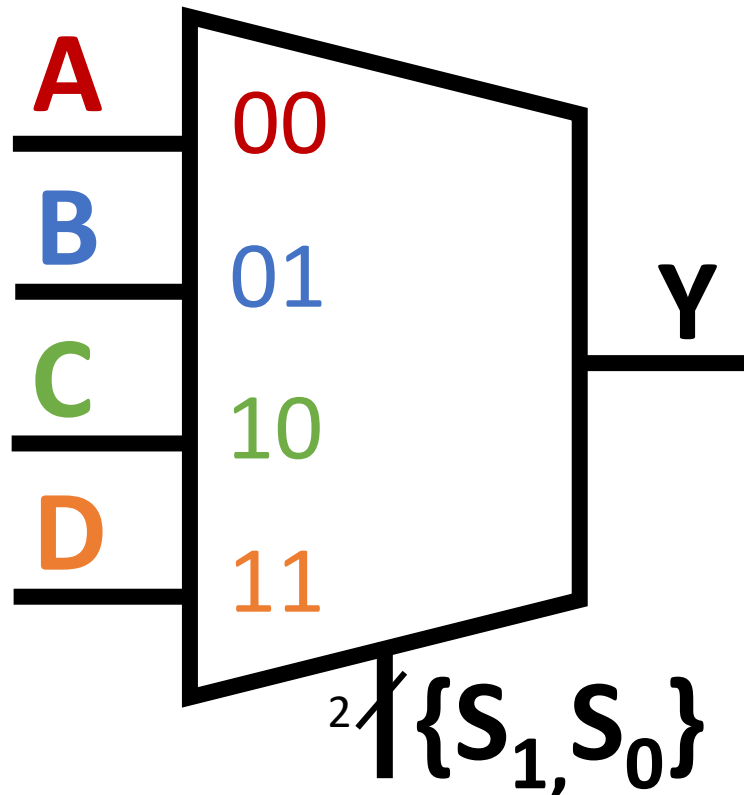
2-to-1 mux



If $S = 0$, $Y = A$

If $S = 1$, $Y = B$

4-to-1 mux

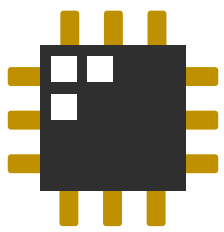


If $\{S_1, S_0\} = 00$, $Y = A$

If $\{S_1, S_0\} = 01$, $Y = B$

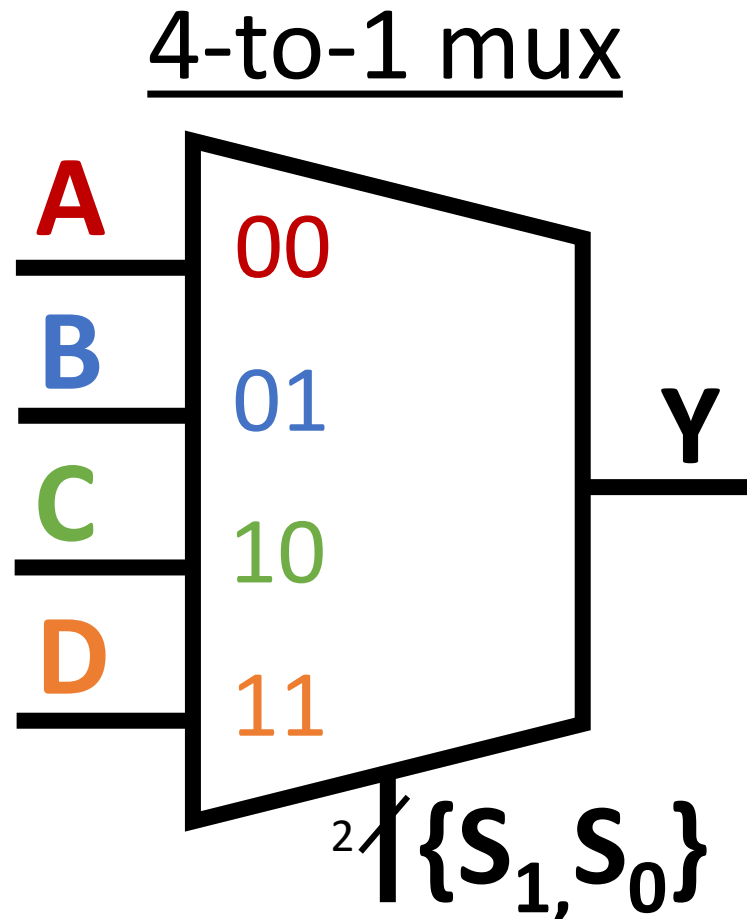
If $\{S_1, S_0\} = 10$, $Y = C$

If $\{S_1, S_0\} = 11$, $Y = D$



Multiplexers

- Write the equivalent Boolean expression for Y.

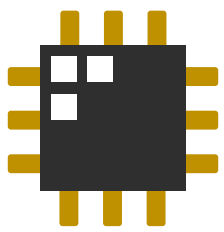


If $\{S_1, S_0\} = 00$, $Y = A$

If $\{S_1, S_0\} = 01$, $Y = B$

If $\{S_1, S_0\} = 10$, $Y = C$

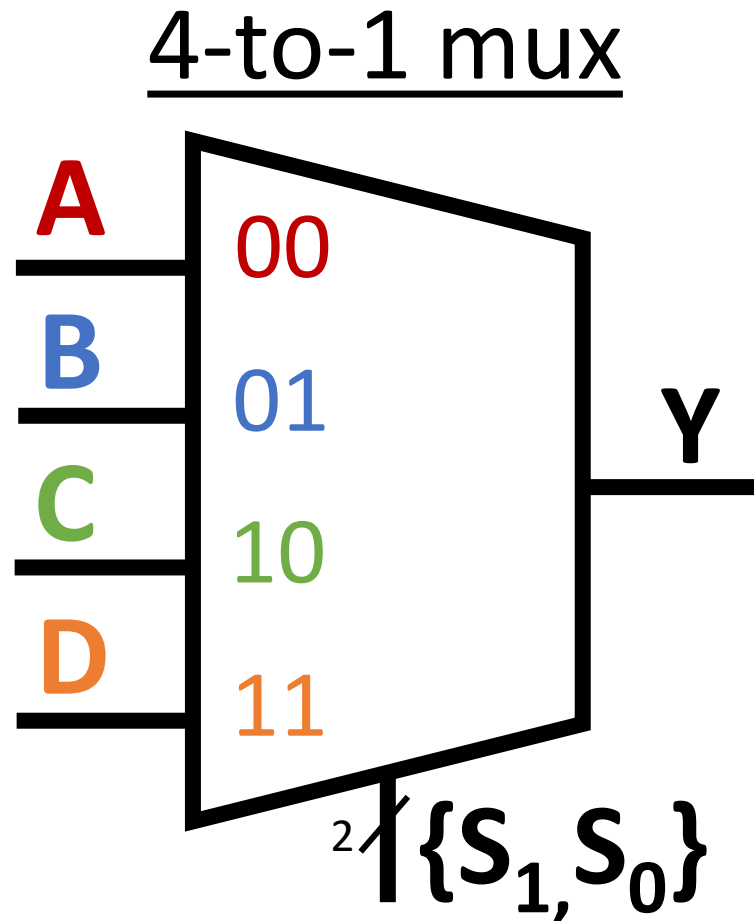
If $\{S_1, S_0\} = 11$, $Y = D$



Multiplexers

- Write the equivalent Boolean expression for Y.

$$Y = S_1'S_0'A + S_1'S_0'B + S_1S_0'C + S_1S_0'D$$

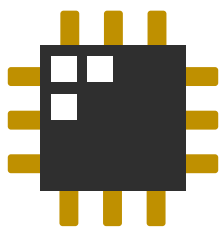


If $\{S_1, S_0\} = 00$, $Y = A$

If $\{S_1, S_0\} = 01$, $Y = B$

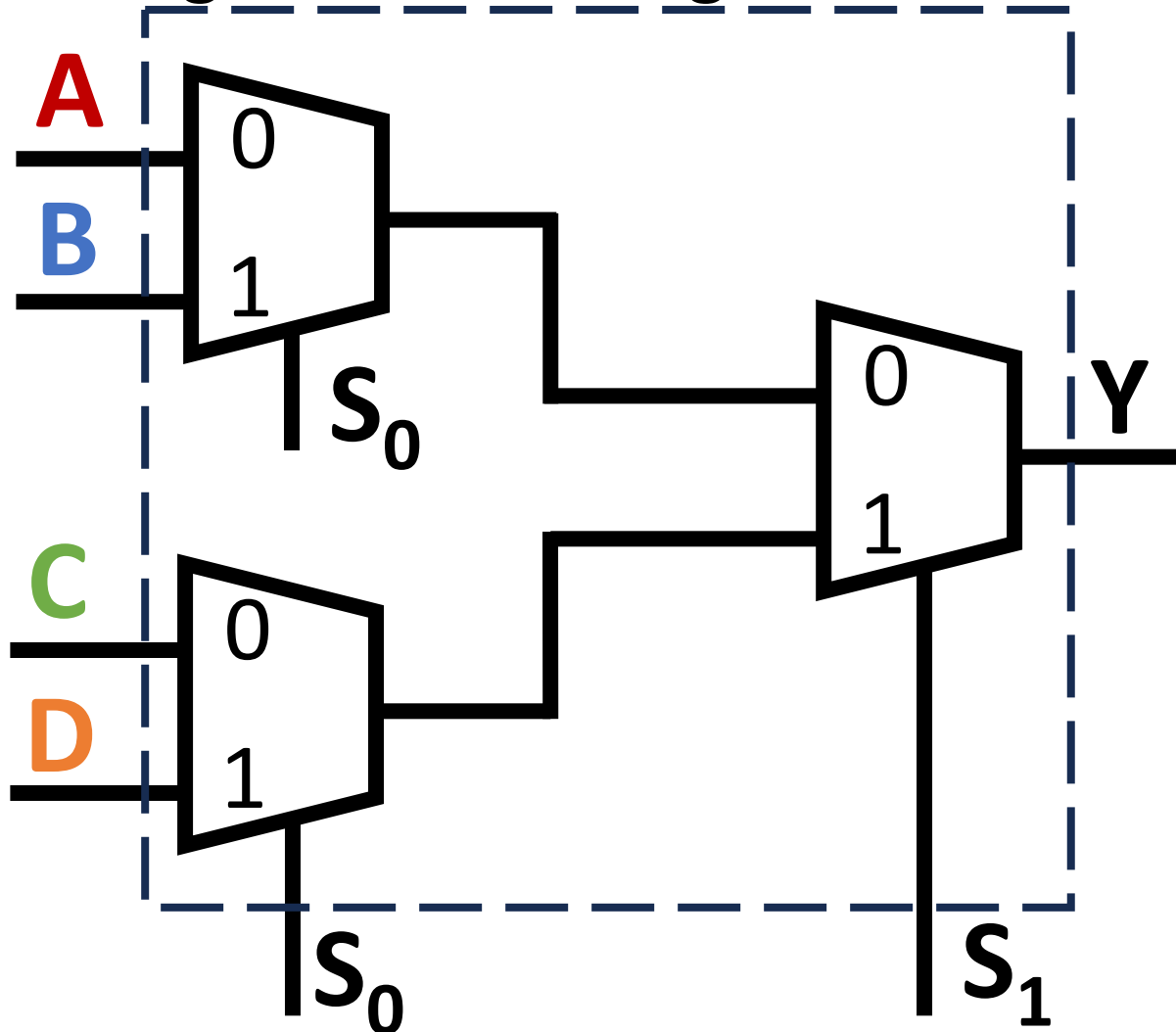
If $\{S_1, S_0\} = 10$, $Y = C$

If $\{S_1, S_0\} = 11$, $Y = D$



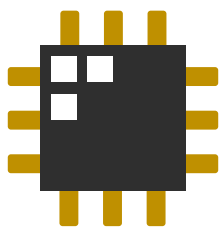
Combining multiplexers

- You can make larger muxes using smaller muxes!



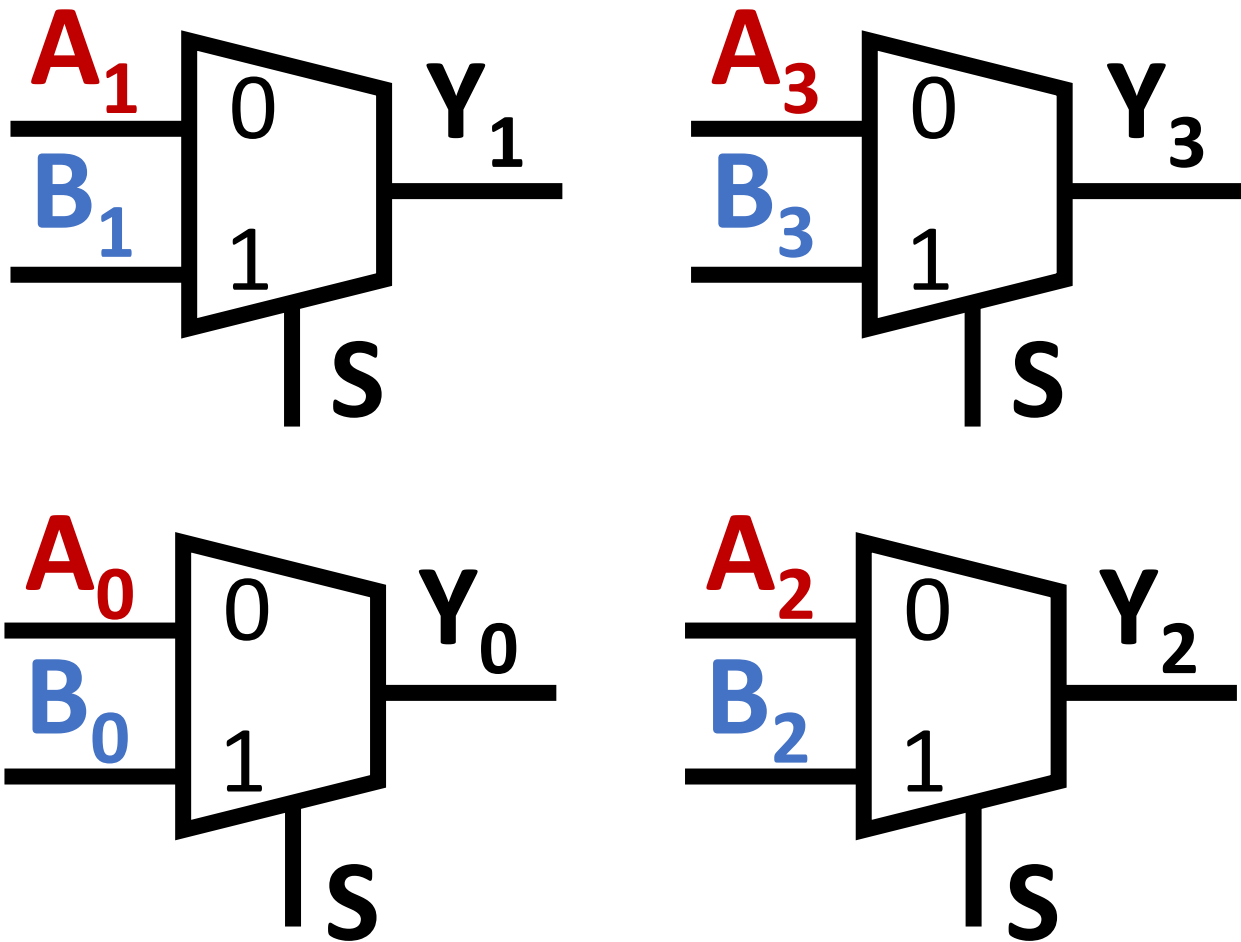
4-to-1 mux using
2-to-1 muxes

If $\{S_1, S_0\} = 00$, $Y = A$
If $\{S_1, S_0\} = 01$, $Y = B$
If $\{S_1, S_0\} = 10$, $Y = C$
If $\{S_1, S_0\} = 11$, $Y = D$



Combining multiplexers

- Multiplexers can be used in parallel to handle several bits at once!

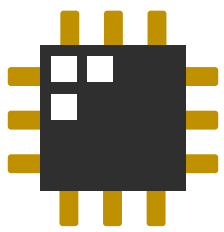


4-bit 2-to-1 mux

If $S = 0$, $\{Y_3, Y_2, Y_1, Y_0\} = \{A_3, A_2, A_1, A_0\}$

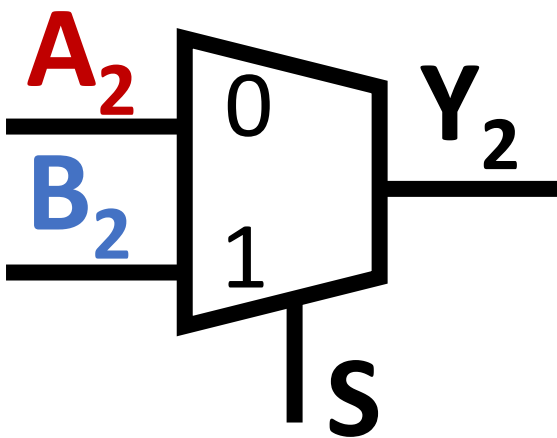
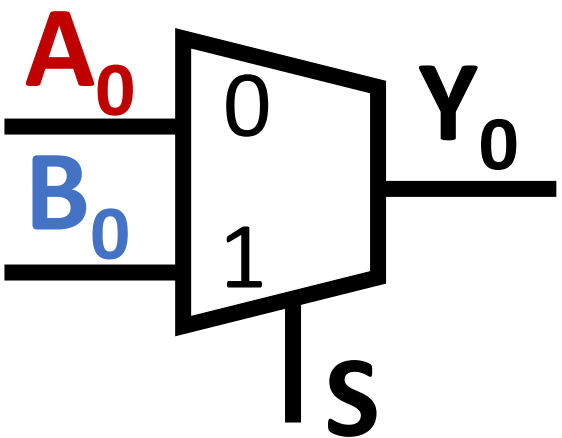
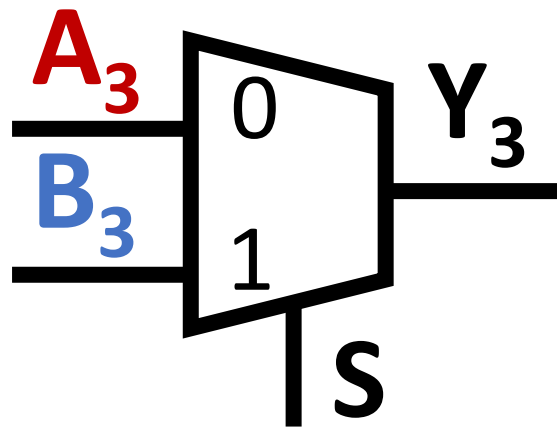
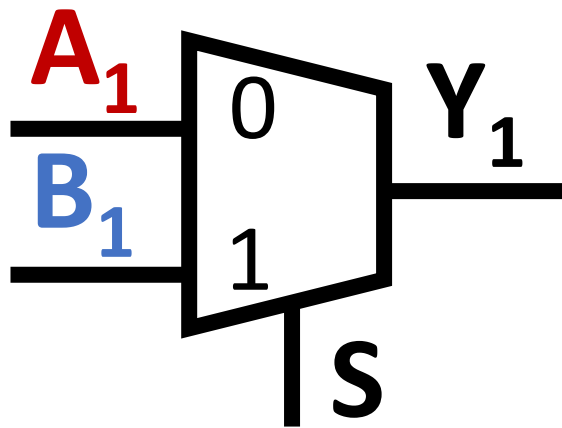
If $S = 1$, $\{Y_3, Y_2, Y_1, Y_0\} = \{B_3, B_2, B_1, B_0\}$

All muxes here *run* in parallel



Combining multiplexers

- Multiplexers can be used in parallel to handle several bits at once!

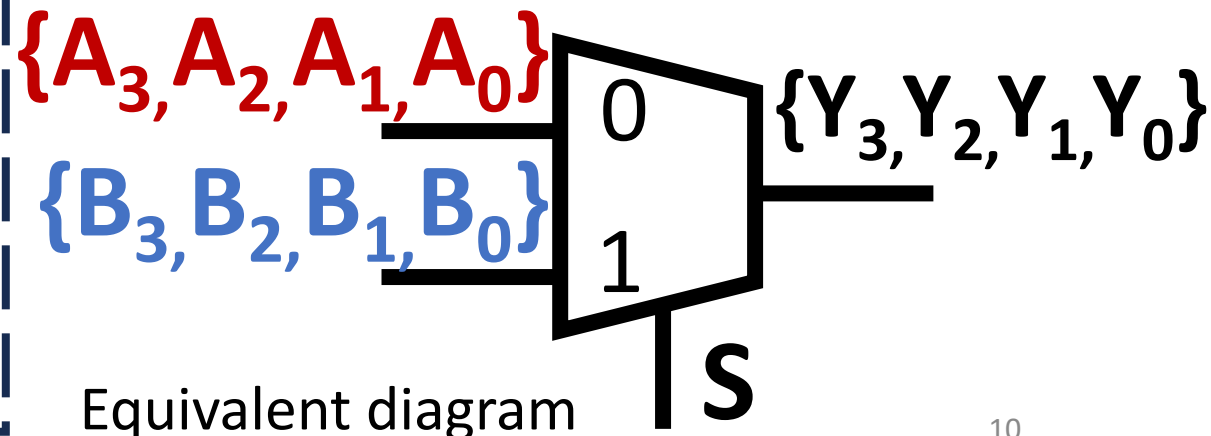


4-bit 2-to-1 mux

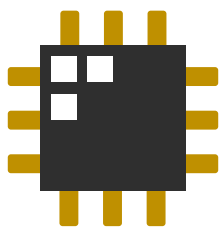
If $S = 0$, $\{Y_3, Y_2, Y_1, Y_0\} = \{A_3, A_2, A_1, A_0\}$

If $S = 1$, $\{Y_3, Y_2, Y_1, Y_0\} = \{B_3, B_2, B_1, B_0\}$

All muxes here *run* in parallel



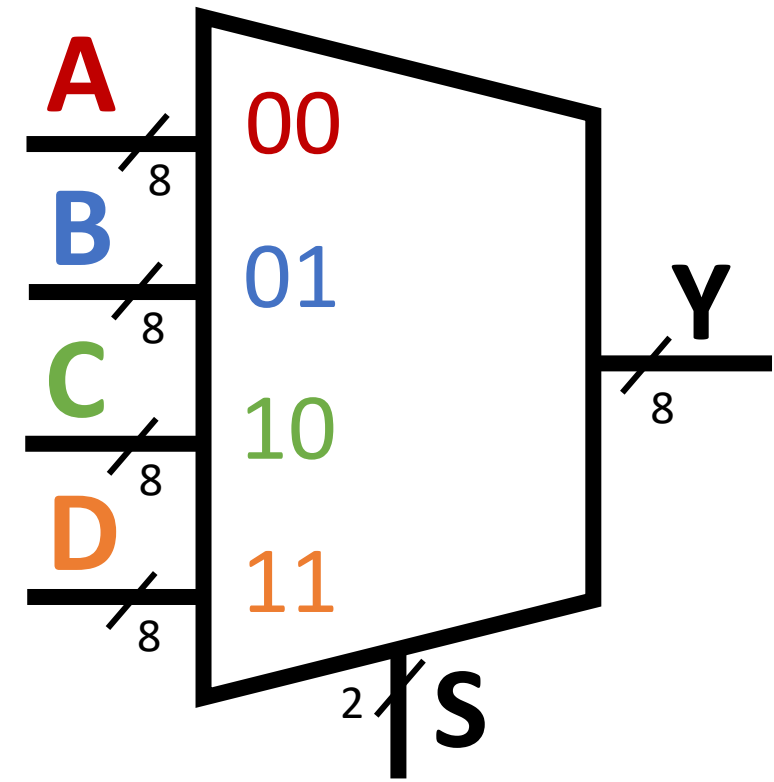
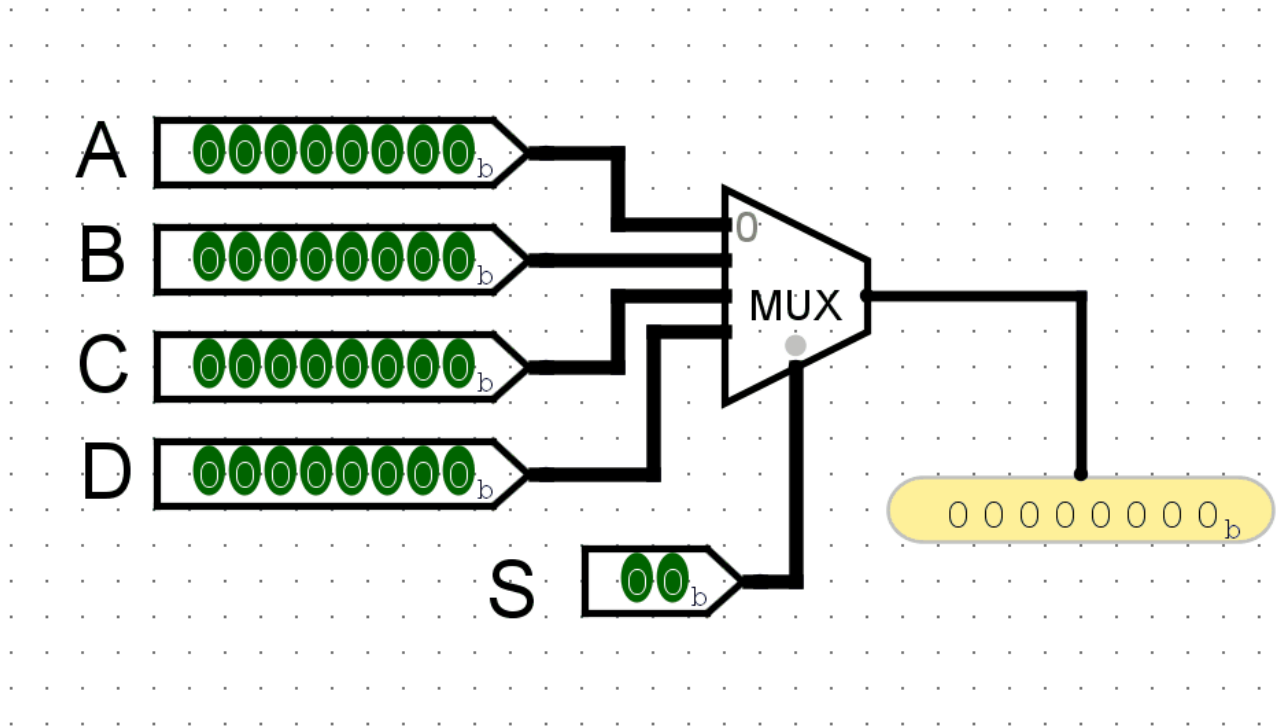
Equivalent diagram



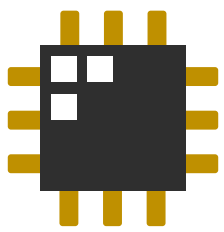
Multiplexers

- Multiplexers can be used in parallel to handle several bits at once!

8-bit 4-to-1 mux



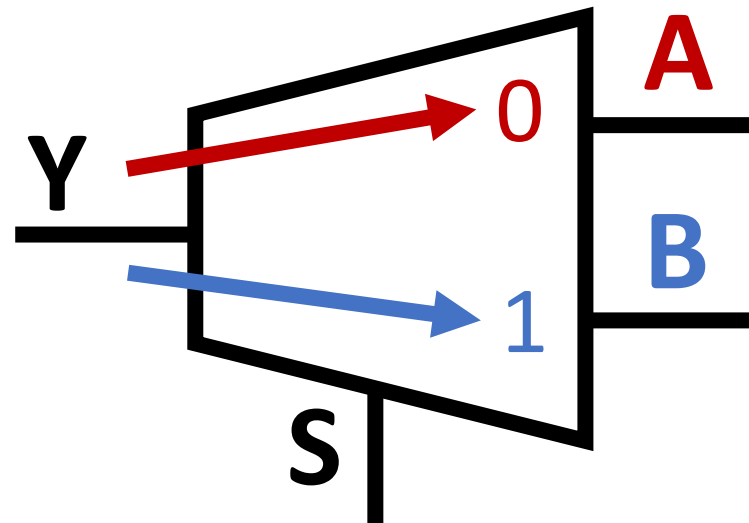
When considering multi-bit muxes, we sometimes don't put the number of bits to make the drawing simpler.



Demultiplexers

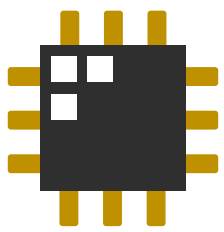
- We can technically do the reverse functionality!

Depending on the selector, direct Y to A or direct Y to B



Inputs		Outputs	
S	Y	A	B
0	0	0	0
0	1	1	0
1	0	0	0
1	1	0	1

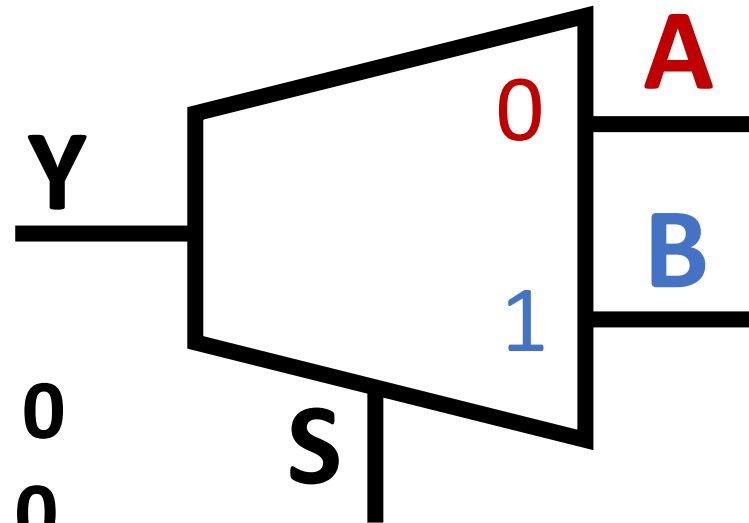
Note: Y is the input here (S is still an input), A and B are outputs



Demultiplexers

- Demultiplexers direct the *single* input to *one of many* (2^n) possible outputs using an n -bit selector (basically the inverse of a multiplexer!)

1-to-2 demux



$$A = S'Y$$

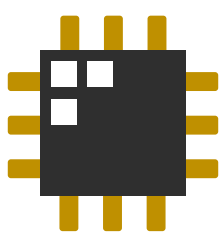
$$B = SY$$

Boolean expressions
for the outputs

If $S = 0$, $A = Y$, else $A = 0$

If $S = 1$, $B = Y$, else $B = 0$

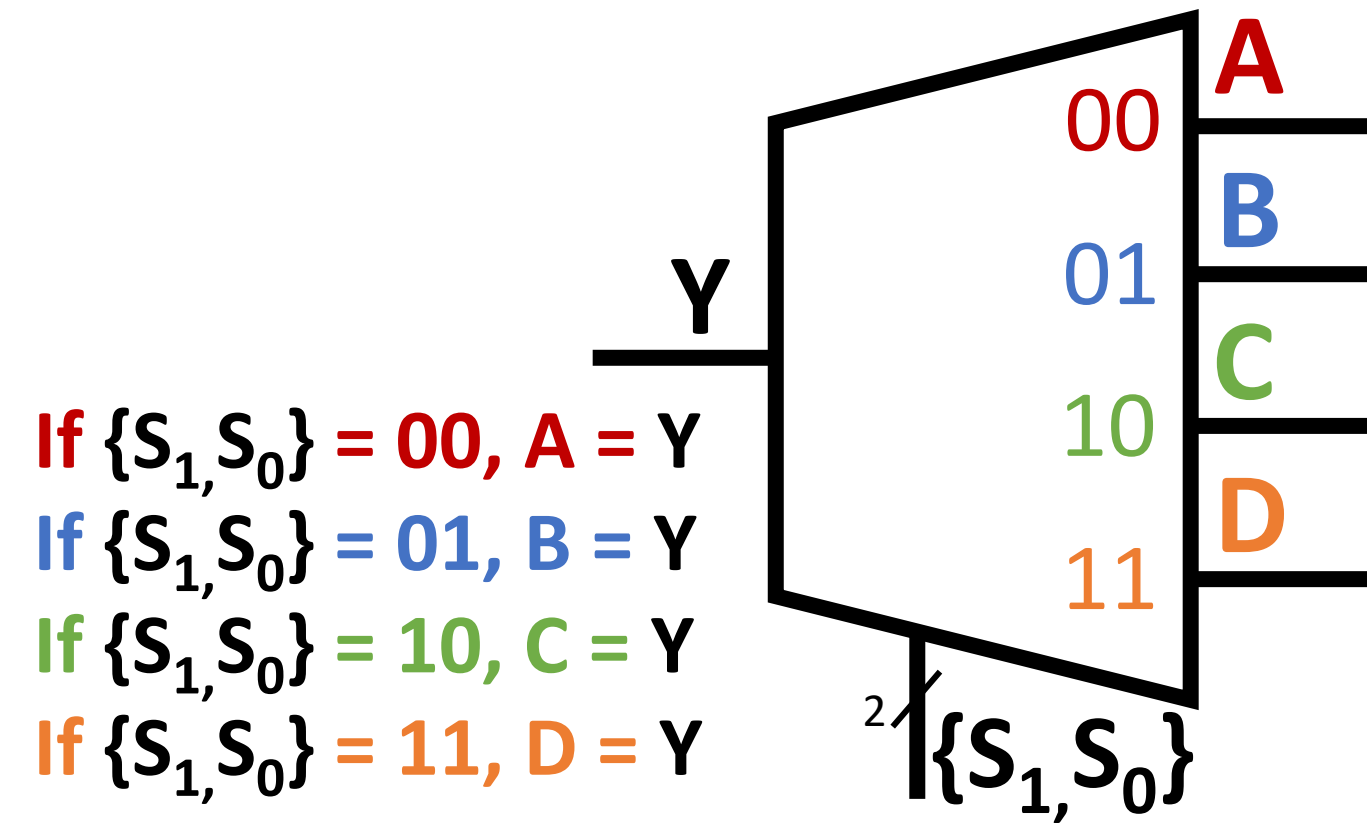
Note: Y is the input here (S is still an input), A and B are outputs



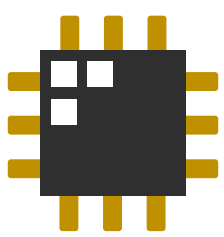
Demultiplexers

- Demultiplexers direct the single input to many (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



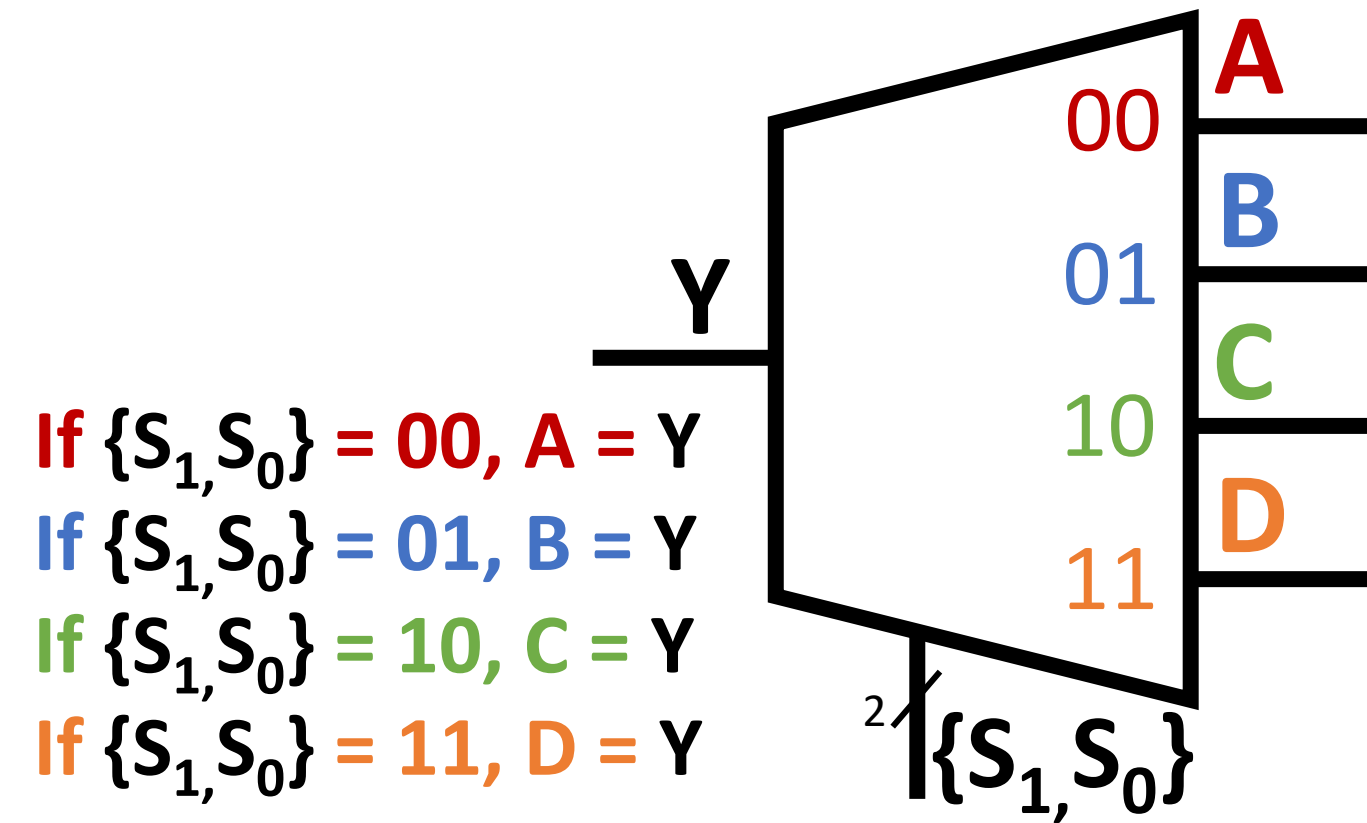
Inputs			Outputs			
S_1	S_0	Y	A	B	C	D
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				



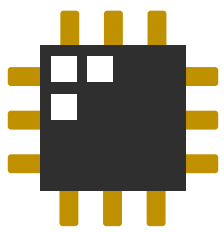
Demultiplexers

- Demultiplexers direct the single input to many (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



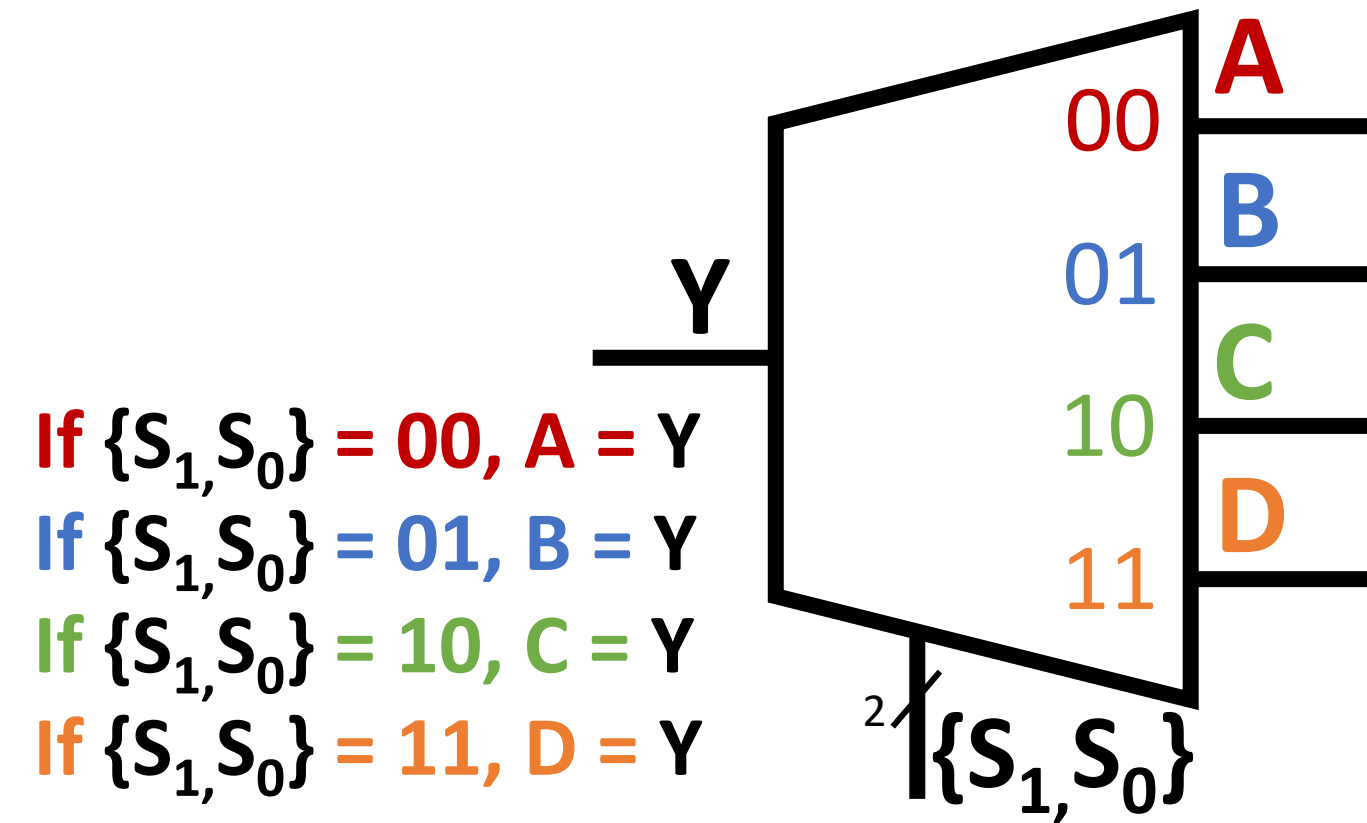
Inputs			Outputs			
S_1	S_0	Y	A	B	C	D
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	0	0
0	1	1	0	1	0	0
1	0	0	0	0	0	0
1	0	1	0	0	1	0
1	1	0	0	0	0	0
1	1	1	0	0	0	1



Demultiplexers

- Demultiplexers direct the single input to many (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



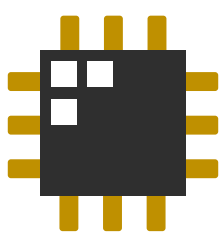
$$A = S_1' S_0' Y$$

$$B = S_1' S_0 Y$$

$$C = S_1 S_0' Y$$

$$D = S_1 S_0 Y$$

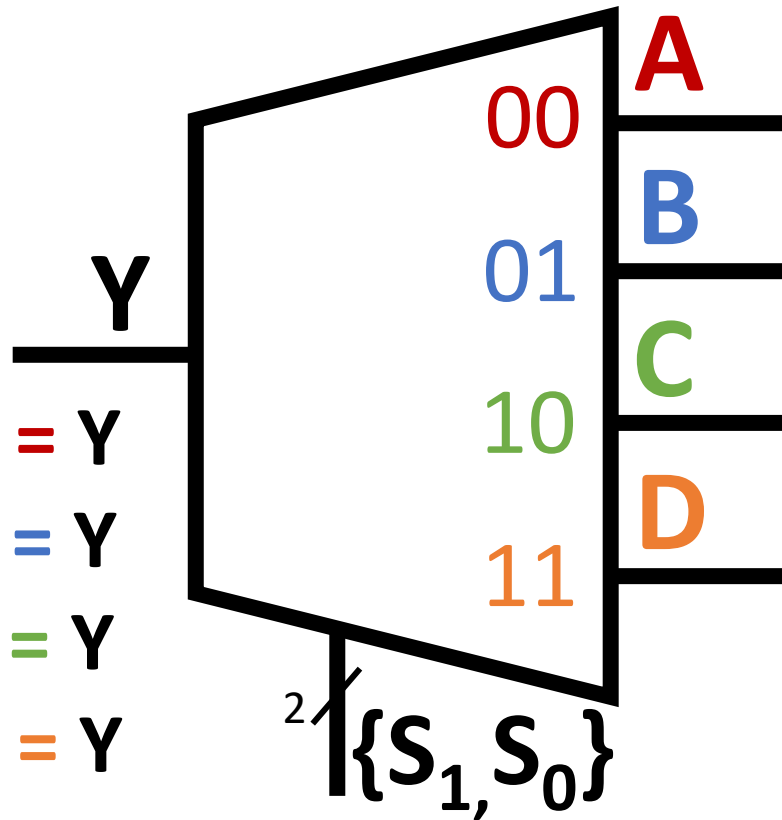
Boolean expressions for the outputs



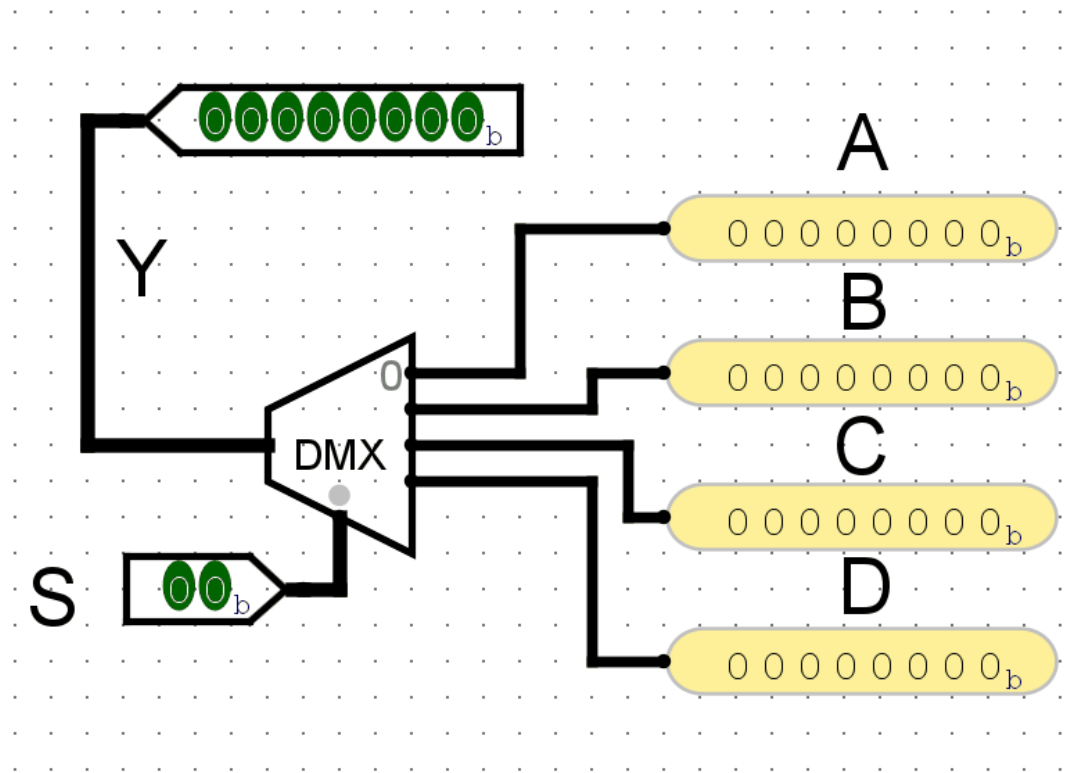
Demultiplexers

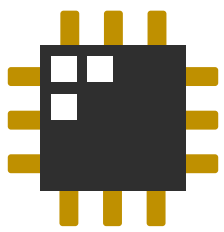
- Demultiplexers direct the single input to many (2^n) possible outputs using an n-bit selector (basically the inverse of a multiplexer!)

1-to-4 demux



If $\{S_1, S_0\} = 00$, **A** = Y
If $\{S_1, S_0\} = 01$, **B** = Y
If $\{S_1, S_0\} = 10$, **C** = Y
If $\{S_1, S_0\} = 11$, **D** = Y



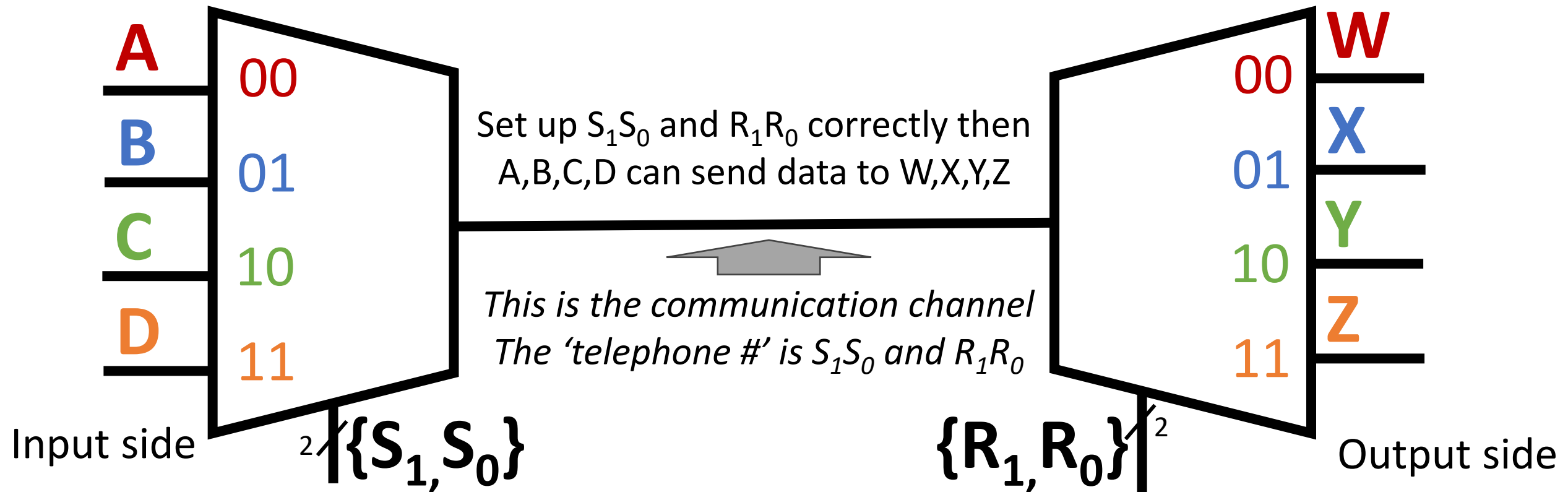


Multiplexer + Demultiplexer

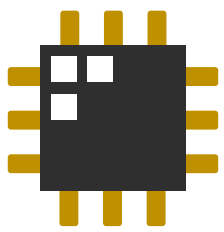
- Aside: Multiplexer + Demultiplexer is like a communication channel

4-to-1 mux

1-to-4 demux



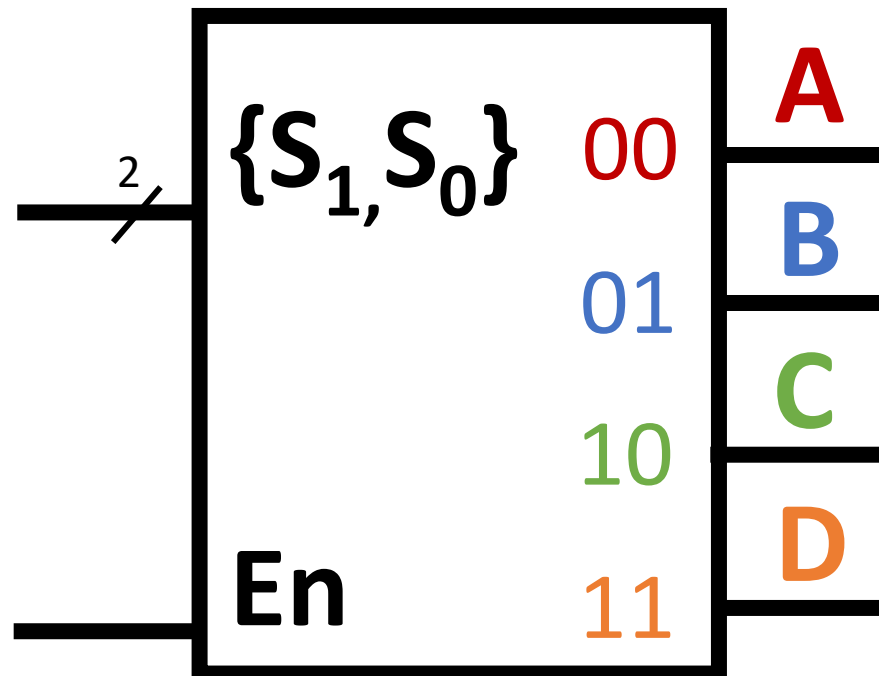
Note: This is a 1-way communication though ☹



Decoders

- Demultiplexers can also be called as *decoders (with enable)*
 - Drawn differently but the functionality is exactly the same!

2-to-4 decoder



The 'primary' input becomes S_1S_0 , thus the block name is 2-to-4 decoder in this example

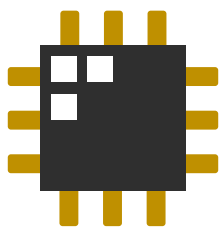
If $\{S_1, S_0\} = 00$, $A = En$

If $\{S_1, S_0\} = 01$, $B = En$

If $\{S_1, S_0\} = 10$, $C = En$

If $\{S_1, S_0\} = 11$, $D = En$

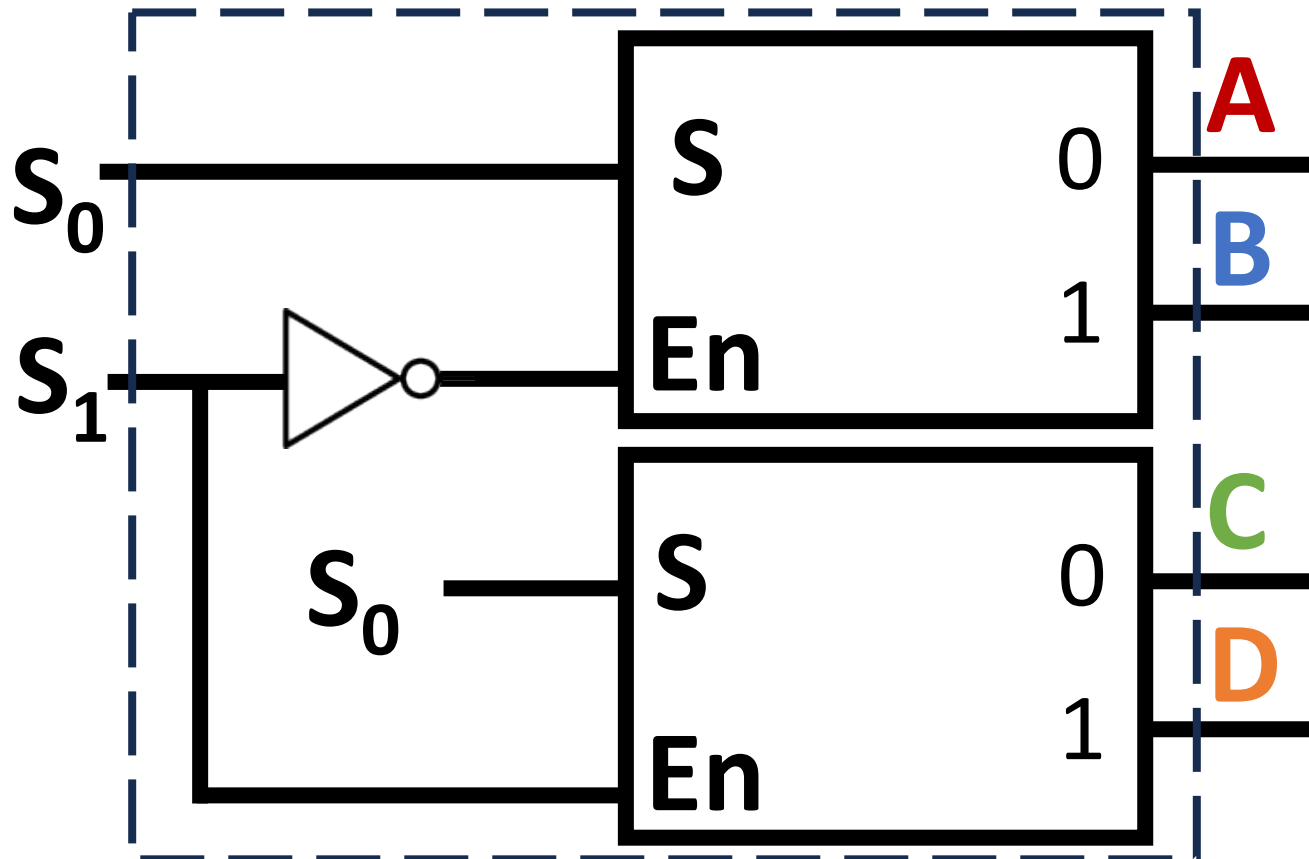
En means enable since the output is *enabled* (outputs a 1) when $En = 1$, else all outputs = 0.



Combining decoders

- You can build larger decoders using smaller ones!

2-to-4 decoder using 1-to-2 decoders



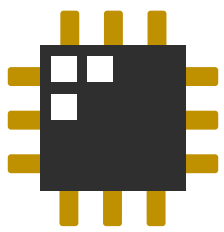
If $\{S_1, S_0\} = 00$, **A** = 1, else 0

If $\{S_1, S_0\} = 01$, **B** = 1, else 0

If $\{S_1, S_0\} = 10$, **C** = 1, else 0

If $\{S_1, S_0\} = 11$, **D** = 1, else 0

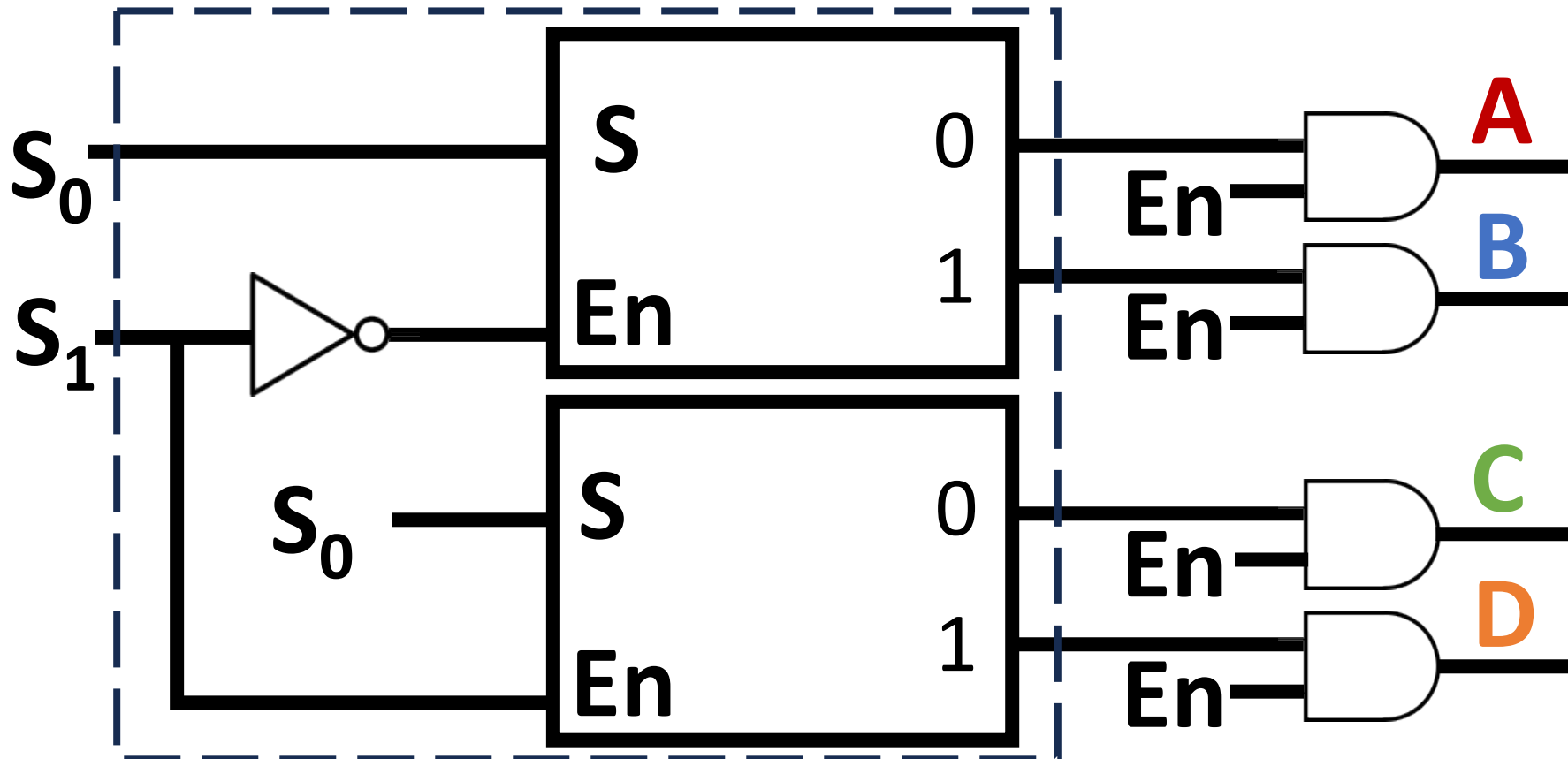
This 2-to-4 decoder does not have an enable though... can you make one?

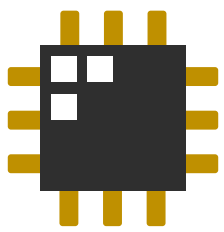


Combining decoders

- Put an enable input for this larger decoder

2-to-4 decoder using 1-to-2 decoders

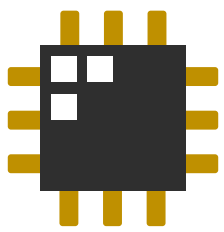




Example circuit using decoders

- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

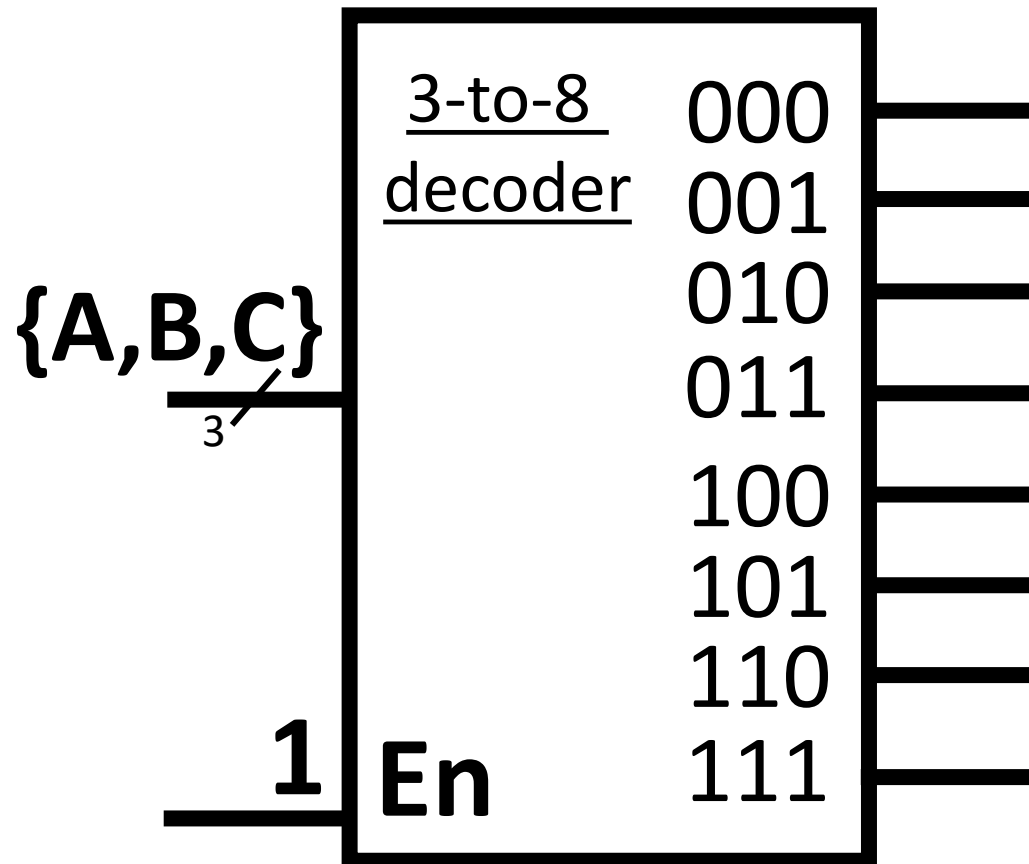
Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

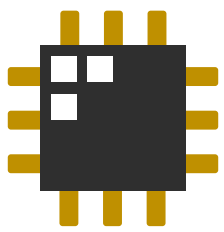


Example circuit using decoders

- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

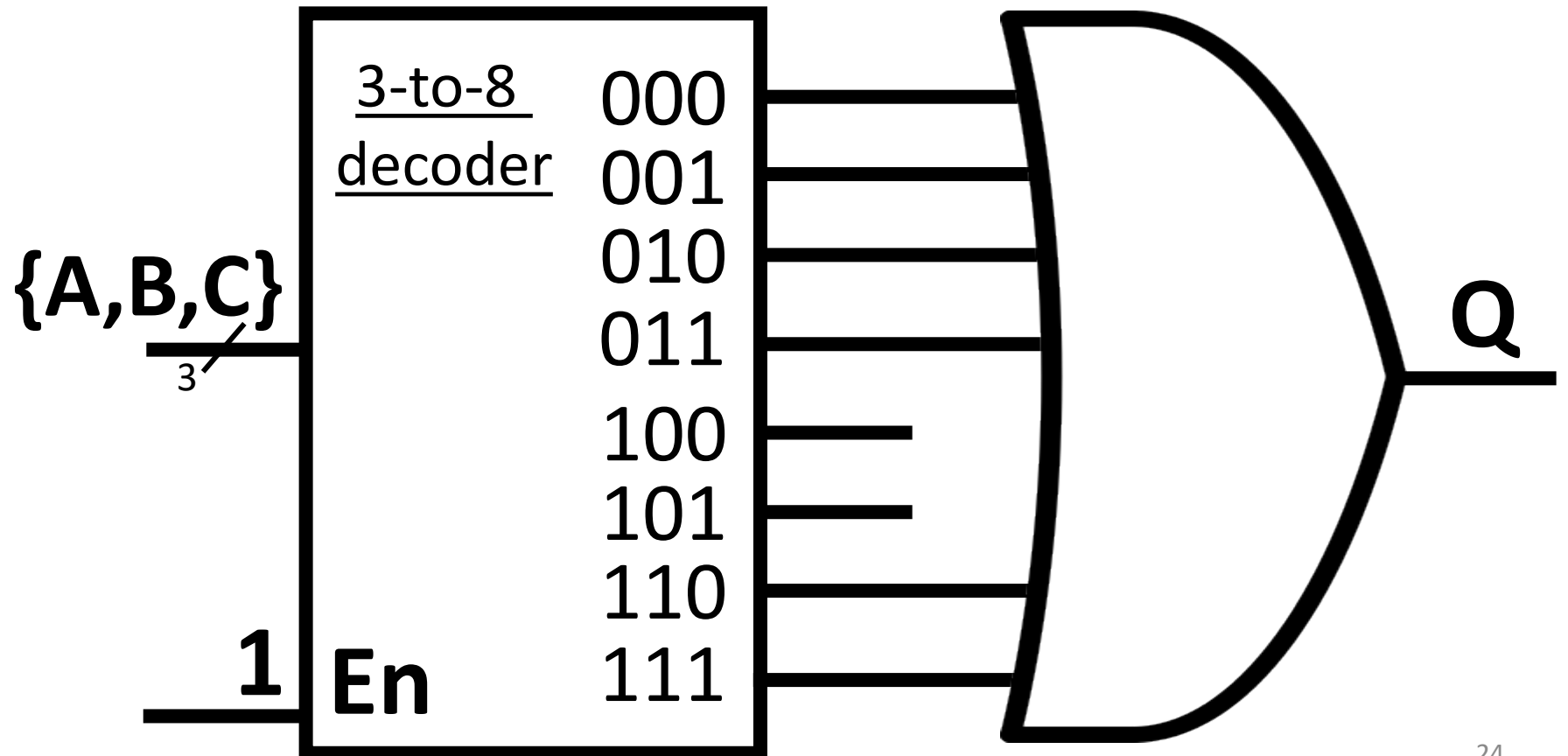


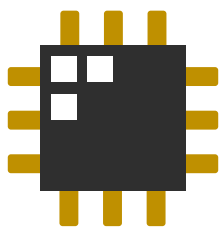


Example circuit using decoders

- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

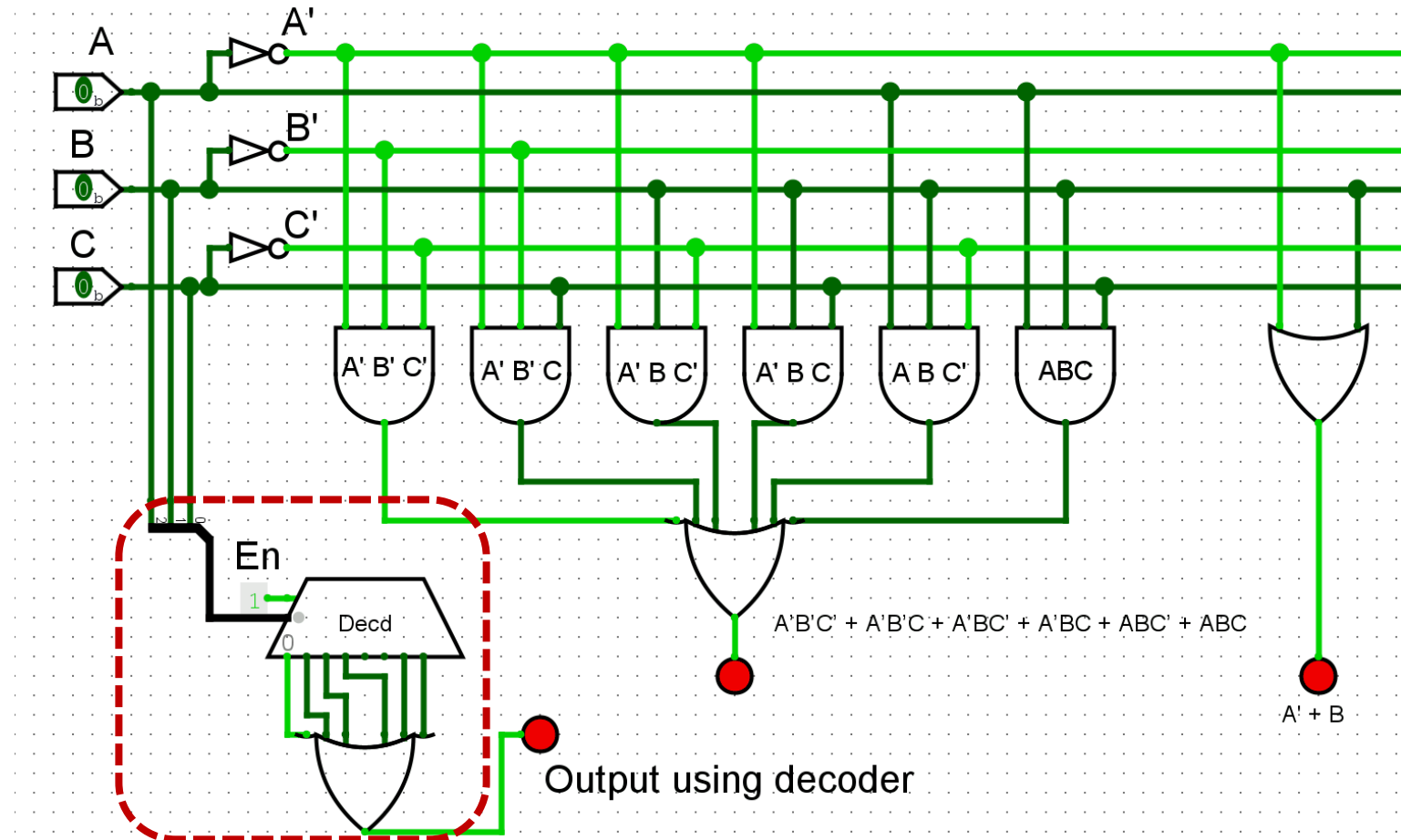


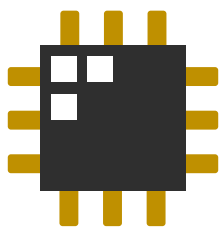


Example circuit using decoders

- Decoders (as well as demultiplexers) allow for an *easy* implementation of truth tables. The outputs are already the per-row combinations!

Inputs			Out
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

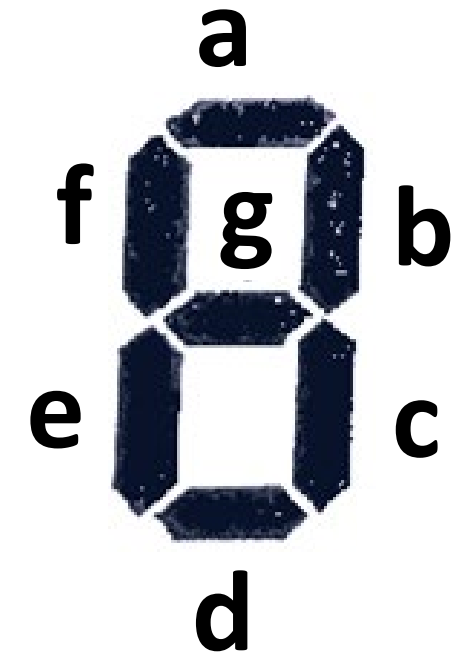




Example circuit using decoders

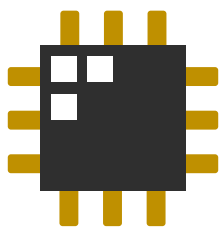
- Fill the table for the 7-segment decoder

Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0							
0	0	0	1							
0	0	1	0							
0	0	1	1							
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1							



Challenge: (inverted logic)

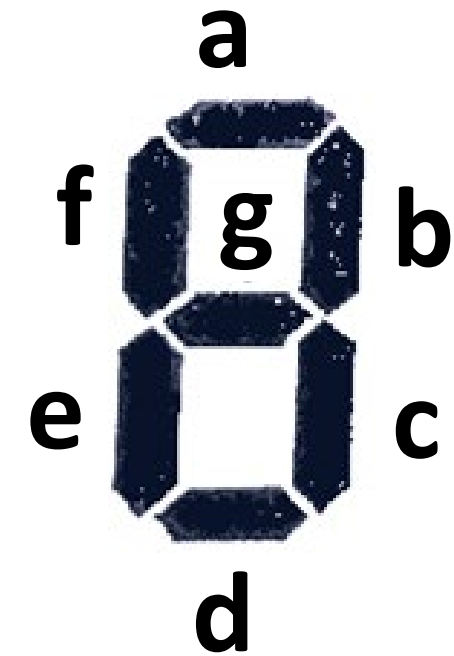
Set output to 0 if you want the LED to be on.



Example circuit using decoders

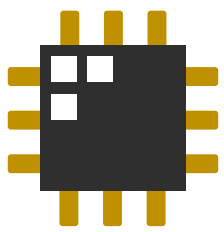
- Fill the table for the 7-segment decoder

Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0
0	0	1	1	0	0	0	0	1	1	0
0	1	0	0	1	0	0	1	1	0	0
0	1	0	1	0	1	0	0	1	0	0
0	1	1	0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	1	0	0



Challenge: (inverted logic)

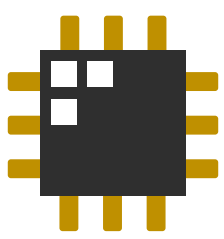
Set output to 0 if you want the LED to be on.



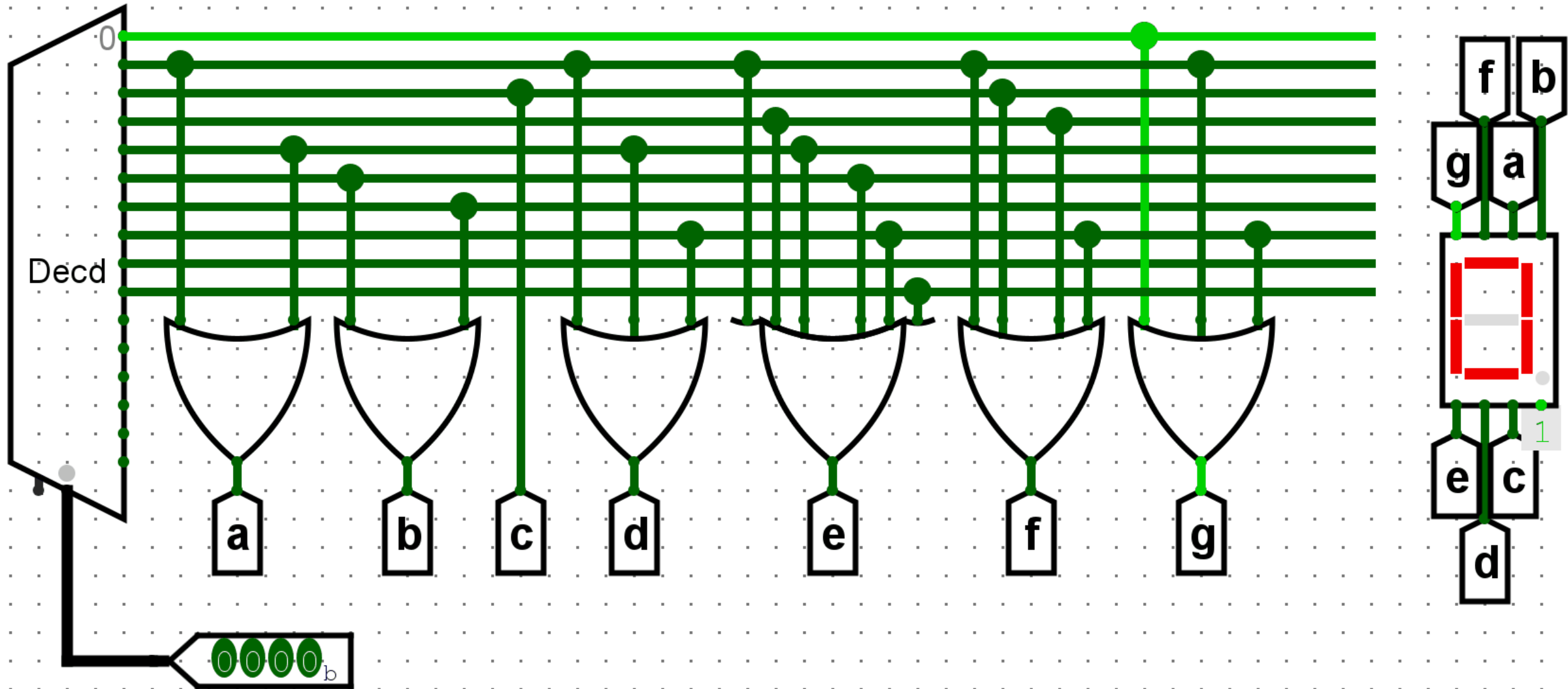
Example circuit using decoders

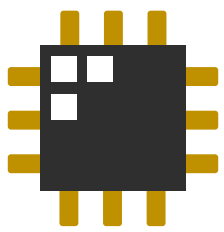
- Using a decoder and OR gates, implement the seven-segment decoder

Inputs				Outputs						
W	X	Y	Z	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0
0	0	1	1	0	0	0	0	1	1	0
0	1	0	0	1	0	0	1	1	0	0
0	1	0	1	0	1	0	0	1	0	0
0	1	1	0	0	1	0	0	0	0	0
0	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	1	0	0



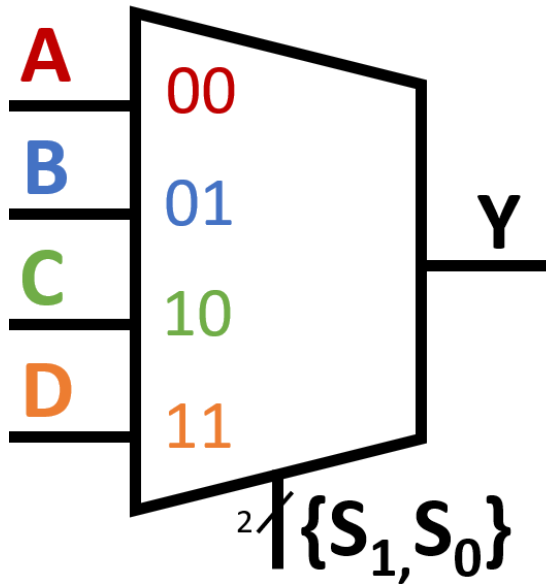
Example circuit using decoders





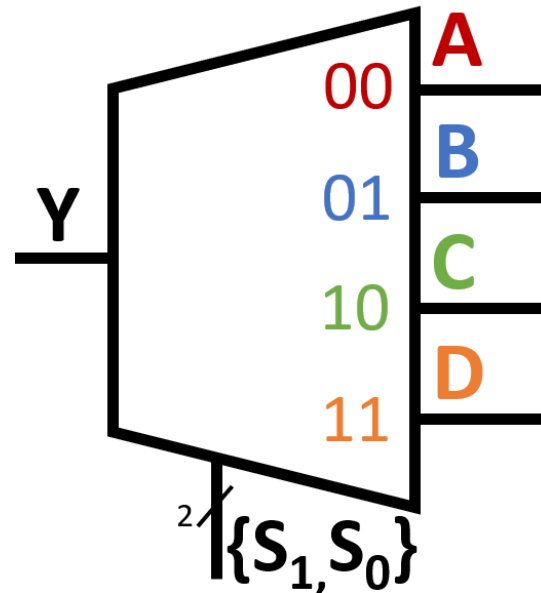
Summary

2^n -to-1 multiplexer



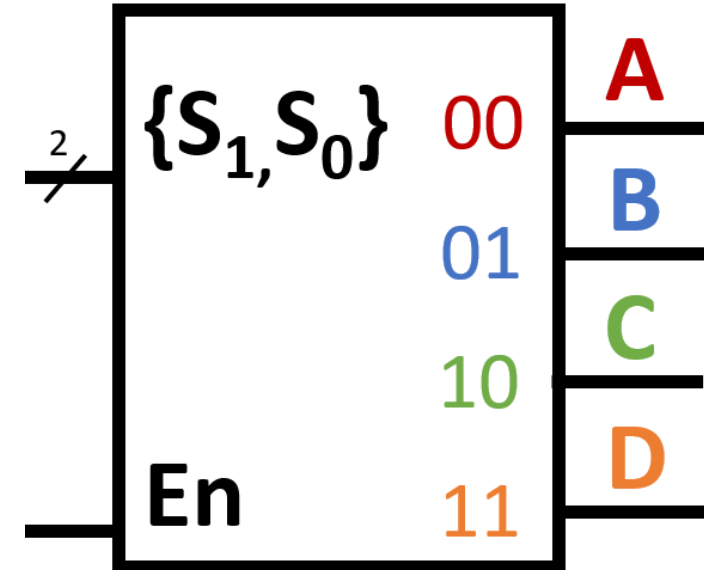
2^n inputs
 n selectors
1 output

1-to- 2^n demultiplexer



1 input
 n selectors
 2^n outputs

n -to- 2^n decoder



n inputs
1 enable
 2^n output